تحليل عملي لـ Insertion Sort على لاعبين FIFA 19 الحقيقيين

زياد محمد علاء الدين | تحليل الخوارزميات | د. محمد عبدالفتاح | المحاضرة الأولى

الفكرة

المشروع بيقيس سرعة Insertion Sort عملياً. استخدمت 2,000 لاعب حقيقيين من داتا FIFA 19، لكل لاعب اسم وتقييم Overall (من 46 لـ 94)، وبرتبهم من الأضعف للأقوى. الداتا في `players.csv` والكود في `insertion_sort_lab.py`.

التجارب

جربت 3 حالات زي سلايد 44:

- **Sorted:** اللاعبين مترتبين من الأول (Best case).
- **Reverse:** ترتيب ملف FIFA الأصلي نفسه، الأقوى الأول (Worst case).
- **Random:** اللاعبين مخلوطين (Average case).

الأحجام: n = 100 و250 و500 و1000 و2000. كل تجربة اتكررت 7 مرات وأخدت أقل زمن، وبعد كل تشغيل اتأكدت إن القايمة مترتبة فعلاً. وقارنت الحالة العشوائية بـ Merge Sort.

النتائج (n = 2000)

الحالة	Comparisons	Shifts	الزمن (ms)
Sorted	1,999	0	0.25
Random	961,147	959,156	60.6
Reverse	1,918,658	1,916,701	116.9

تطابق النظرية: في الـ Best case العدد طلع بالظبط (1,999) n−1. في الـ Worst case طلع 1,918,658، وهو أقل من n(n−1)/2 = 1,999,000 بحوالي 4%، لأن اللاعبين اللي ليهم نفس التقييم مبيتزحزحوش. الـ Random طلع حوالي نص الـ Worst case، يعني قريب من $n^2/4$.

اختبار Scaling: لما n اتضاعفت من 1000 لـ 2000، زمن الـ Reverse زاد من 29.4 لـ 116.9 ms، يعني حوالي 4 مرات، وده المتوقع من $O(n^2)$. وفي الـ Sorted عدد المقارنات اتضاعف بس (999 لـ 1,999)، وده $O(n)$.

المقارنة مع Merge Sort (Random)

n	Insertion (ms)	Merge (ms)
500	4.49	0.45
1000	14.9	0.79
2000	60.6	3.60

عند 2000 لاعب Merge Sort أسرع بحوالي 17 مرة، والفرق بيكبر مع n.

الـ Time-Space Trade-off

Insertion Sort بيشتغل in-place على القوايم الكبيرة ومش محتاج ذاكرة إضافية، بس بطيء. Merge Sort بيعمل نسخ جديدة في كل دمج، فبياخد ذاكرة أكتر عشان ياخد وقت أقل.

الخلاصة

- Insertion Sort: Worst $O(n^2)$ و Best $\Omega(n)$ و Average $\Theta(n^2)$، والتجربة أكدت التلاتة.
- عدد الـ Shifts هو اللي بيتحكم في الوقت: صفر في المترتبة وحوالي 1.9 مليون في الـ Reverse.
- Merge Sort أحسن مناسب للقوايم الصغيرة أو القريبة من الترتيب، وللقوايم الكبيرة العشوائية.

الرسم في الملف `result.png`، ضيفه في التقرير.